Date:-15/10/2026

Week-3

Ant Colony Optimization for the Traveling Salesman Problem:

Pseudocode

START

1. Initialize Cities

   Define the coordinates of all cities.

   Calculate the distance between every pair of cities.

2. Initialize Parameters

   Set number of ants (num_ants)

   Set alpha (pheromone importance)

   Set beta (heuristic importance)

   Set rho (evaporation rate)

   Set initial pheromone value

   Set maximum number of iterations

   Initialize pheromone matrix.

   Set best_distance = INFINITY

3. FOR each iteration DO

   FOR each ant DO

   Select a random starting city.

   Mark the starting city as visited.

   WHILE all cities are not visited DO

   Calculate the probability of selecting

   each unvisited city.

   Probability formula:

P(i,j) = [pheromone(i,j)^alpha *(1/distance(i,j))^beta] / Sum of all unvisited cities

Select the next city based on probability.

Add the selected city to the tour.

Mark the selected city as visited.

END WHILE

Return to the starting city.

Calculate the total tour distance.

IF tour distance < best_distance THEN

best_distance = tour distance

best_tour = current tour

END IF

END FOR

Update Pheromones:

1. Evaporate pheromones: pheromone(i,j) = (1-rho) * pheromone(i,j)

2. Deposit new pheromones: pheromone(i,j) = pheromone(i,j) + Q / tour_distance

where Q is a constant.

END FOR

4. Display Results

Print best_tour.

Print best_distance.

STOP

Python code : Ant Colony Optimization (ACO) for the Traveling Salesman Problem (TSP)

```
import math
import random
import matplotlib.pyplot as plt

# STEP 1: Define cities with coordinates
cities = {
    0: (0, 0),
    1: (2, 3),
    2: (5, 2),
    3: (6, 6),
    4: (8, 3),
    5: (1, 7),
    6: (4, 8),
    7: (9, 7)
}
n = len(cities)

# STEP 2: Initialize ACO parameters

num_ants = 100
num_iterations = 100
alpha = 1.0
beta = 1.0
rho = 0.05
Q = 1.0
```

```
random.seed(42)

# STEP 3: Calculate distance
def calculate_distance(city1, city2):
    x1, y1 = cities[city1]
    x2, y2 = cities[city2]
    return math.sqrt((x1-x2)**2 + (y1-y2)**2)

distance = [
    [calculate_distance(i, j) for j in range(n)]
    for i in range(n)
]

# STEP 4: Initialize pheromones

def nearest_neighbor_length():
    unvisited = set(range(n))
    current = 0
    unvisited.remove(current)
    total = 0

    while unvisited:
        next_city = min(
            unvisited,
            key=lambda city: distance[current][city]
        )
        total += distance[current][next_city]
        current = next_city
        unvisited.remove(current)
```

```
    total += distance[current][0]
    return total

Lnn = nearest_neighbor_length()
initial_pheromone = 1 / (n * Lnn)

pheromone = [ [initial_pheromone for j in range(n)]
    for i in range(n)
]

# STEP 5: Select next city

def select_next_city(current, unvisited):
    probabilities = []
    for city in unvisited:
        value = (pheromone[current][city] ** alpha) * ((1 / distance[current][city]) ** beta)

        probabilities.append(value)

    total = sum(probabilities)
    probabilities = [p / total for p in probabilities]
    return random.choices(list(unvisited), weights=probabilities, k=1    )[0]

# STEP 6: Construct ant tour

def construct_tour():
    start = random.randint(0, n-1)
    tour = [start]
    unvisited = set(range(n))
    unvisited.remove(start)
```

```
    current = start

    while unvisited:
        next_city = select_next_city(current, unvisited)

        tour.append(next_city)
        unvisited.remove(next_city)

        current = next_city

    return tour

# STEP 7: Calculate total route distance

def calculate_tour_length(tour):

    total = 0

    for i in range(n-1):
        total += distance[tour[i]][tour[i+1]]

    total += distance[tour[-1]][tour[0]]

    return total

# STEP 8: Update pheromones

def update_pheromones(ant_tours, ant_distances):
```

```
    global pheromone

    # Evaporation
    for i in range(n):
        for j in range(n):
            pheromone[i][j] *= (1-rho)

    # Deposit pheromones
    for tour, length in zip(ant_tours, ant_distances):

        deposit = Q / length

        for i in range(n):
            city1 = tour[i]
            city2 = tour[(i+1) % n]

            pheromone[city1][city2] += deposit
            pheromone[city2][city1] += deposit

# STEP 9: Main ACO algorithm

best_tour = None
best_distance = float('inf')
best_distance_history = []

for iteration in range(num_iterations):
    ant_tours = []
    ant_distances = []

    for ant in range(num_ants):
```

```
        tour = construct_tour()
        length = calculate_tour_length(tour)

        ant_tours.append(tour)
        ant_distances.append(length)

        if length < best_distance:
            best_distance = length
            best_tour = tour.copy()

    update_pheromones(ant_tours, ant_distances)
    best_distance_history.append(best_distance)

    print("Iteration:", iteration+1, "Best Distance:", round(best_distance, 2) )


# STEP 10: Display result

print("\n========== ACO RESULTS ==========")
route = best_tour + [best_tour[0]]
print("Best Route:", " -> ".join(map(str, route)))
print("Shortest Distance:", round(best_distance, 2))

# STEP 11: Graph 1 - Best Route
plt.figure(figsize=(9, 6))
x = [cities[i][0] for i in route]
y = [cities[i][1] for i in route]
plt.plot(x, y, 'o-', linewidth=2, markersize=8)

for city, (cx, cy) in cities.items():
```

```
    plt.annotate(
        f"City {city}",
        (cx, cy),
        xytext=(5, 7),
        textcoords="offset points"
    )

plt.title("Ant Colony Optimization - Best Route")
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.grid(True)
plt.tight_layout()

# STEP 12: Graph 2 - Convergence
plt.figure(figsize=(9, 6))
plt.plot(
    range(1, num_iterations+1),
    best_distance_history,
    linewidth=2
)

plt.title("ACO Convergence Graph")
plt.xlabel("Number of Iterations")
plt.ylabel("Best Distance")
plt.grid(True)
plt.tight_layout()

# Display both graphs

plt.show()
```

⇨ **OUTPUT**

```
========== ACO RESULTS ==========
Best Route: 6 -> 5 -> 1 -> 0 -> 2 -> 4 -> 7 -> 3 -> 6
Shortest Distance: 29.55
```



